

NoPrumo

Construção passo a passo de uma API REST em ASP.NET Core com Clean Architecture, Entity Framework Core e MySQL.

Domínio	Gestão de clientes para pequenas construtoras
Entidade	Cliente (nome, obras simultâneas, orçamento)
Plataforma	.NET 8 · ASP.NET Core Web API
Persistência	EF Core 8 + Pomelo MySQL
IDE	Visual Studio
Resultado	CRUD completo funcionando via Swagger

Sumário

ESTRUTURA DA SOLUTION

- 01 Criar o projeto Web API
- 02 Criar as bibliotecas de classes
- 03 Definir os RootNamespaces
- 04 Ligar as referências entre projetos
- 05 Limpar o template WeatherForecast

CAMADA 04-DOMAIN

- 06 Entidade `Cliente`
- 07 Interface `IClienteRepository`

CAMADA 02-APPLICATION

- 08 DTOs de entrada e saída
- 09 Interface `IClienteService`
- 10 Implementação `ClienteService`

CAMADA 03-INFRASTRUCTURE

- 11 Pacotes NuGet do EF Core
- 12 `AppDbContext`
- 13 `ClienteRepository`

CAMADA 01-PRESENTATION

- 14 Pacote `EFCore.Design`
- 15 String de conexão
- 16 `Program.cs`
- 17 `CientesController`

BANCO DE DADOS E VALIDAÇÃO

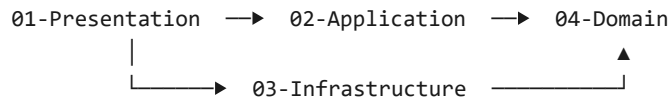
- 18 Gerar a migration
- 19 Criar o banco
- 20 Testar o CRUD no Swagger

APÊNDICES

- A Erros comuns e como resolver
- B Mapa final do projeto

Visão geral da arquitetura

O NoPrumo é organizado em **Clean Architecture**: cinco projetos separados, cada um com uma responsabilidade, e uma regra de dependência que nunca se quebra.



Leia as setas como "*depende de*". Três consequências práticas:

- O **04-Domain** não referencia ninguém. Ele contém a entidade `Cliente` e as interfaces de repositório. Não sabe que existe banco de dados, nem HTTP, nem Entity Framework.
- O **02-Application** não referencia o **03-Infrastructure**. O `ClienteService` depende da interface `IClientRepository`, que mora no Domain. Ele nunca sabe quem vai implementá-la.
- Ninguém referencia o **01-Presentation**. A camada de entrega é a ponta da corrente — pode ser trocada por um app console ou um worker sem afetar o resto.

É o `Program.cs`, no **01-Presentation**, que amarra interface a implementação concreta. Esse é o único ponto do sistema que conhece todo mundo.

Papel de cada projeto

Projeto	Namespace	Contém
01-Presentation	NoPrumo	Controllers, <code>Program.cs</code> , <code>appsettings.json</code>
02-Application	NoPrumo.Application	DTOs, interfaces de serviço, serviços
03-Infrastructure	NoPrumo.Infrastructure	<code>AppDbContext</code> , repositórios, migrations
04-Domain	NoPrumo.Domain	Entidades e interfaces de repositório
05-Tests	NoPrumo.Tests	Testes com xUnit (não usado nesta etapa)

POR QUE OS NÚMEROS NAS PASTAS

Os prefixos `01-` a `05-` servem só para o Solution Explorer listar as camadas na ordem do fluxo de uma requisição. Eles são nome de *pasta*, não de *namespace* — e por isso cada projeto declara o próprio `RootNamespace` no passo 3.

Parte 1 — Estrutura da solution

01 Criar o projeto Web API

No Visual Studio: Criar um novo projeto → ASP.NET Core Web API (C#).

Nome do projeto	01-Presentation
Nome da solution	NoPrumo
Framework	.NET 8.0
Usar controladores	marcado
Habilitar OpenAPI	marcado
HTTPS	marcado

Marque "Usar controladores". Se deixar desmarcado, o template gera *minimal APIs* — endpoints escritos direto no `Program.cs` — e o passo 17 não encaixa.

02 Criar as bibliotecas de classes

Para cada um dos quatro projetos abaixo: botão direito na **Solution** → **Adicionar** → **Novo Projeto**.

Nome	Modelo
02-Application	Biblioteca de Classes (C#)
03-Infrastructure	Biblioteca de Classes (C#)
04-Domain	Biblioteca de Classes (C#)
05-Tests	Projeto de Teste xUnit (C#)

Todos em **.NET 8.0**, no mesmo local da solution. Em seguida apague o `Class1.cs` (e o `UnitTest1.cs`) que cada template gera.

03 Definir os RootNamespaces

Dê duplo clique em cada projeto para abrir o `.csproj` e ajuste a linha `<RootNamespace>` dentro do `<PropertyGroup>` :

```
<!-- 01-Presentation.csproj -->
<RootNamespace>NoPrumo</RootNamespace>

<!-- 02-Application.csproj -->
<RootNamespace>NoPrumo.Application</RootNamespace>

<!-- 03-Infrastructure.csproj -->
```

```
<RootNamespace>NoPrumo.Infrastructure</RootNamespace>

<!-- 04-Domain.csproj -->
<RootNamespace>NoPrumo.Domain</RootNamespace>

<!-- 05-Tests.csproj -->
<RootNamespace>NoPrumo.Tests</RootNamespace>
```

POR QUE ISSO É OBRIGATÓRIO

Um namespace em C# não pode começar com número nem conter hífen — `01-Presentation` viola as duas regras. Sem essa linha, o Visual Studio "conserta" sozinho e passa a gerar arquivos com namespace `_01_Presentation`. Seus `using` ficariam assim:

```
using _04_Domain.Entities;    // sem RootNamespace
using NoPrumo.Domain.Entities; // com RootNamespace
```

04 Ligar as referências entre projetos

Em cada projeto: botão direito → **Adicionar** → **Referência de Projeto** → marque os indicados.

Neste projeto	Marque
02-Application	04-Domain
03-Infrastructure	04-Domain
01-Presentation	02-Application e 03-Infrastructure
05-Tests	02-Application e 03-Infrastructure

O `04-Domain` fica sem nenhuma referência. É a inversão de dependência acontecendo na prática: a regra de negócio não conhece a infraestrutura.

05 Limpar o template WeatherForecast

No `01-Presentation`, exclua os dois arquivos de exemplo:

- `WeatherForecast.cs`
- `Controllers\WeatherForecastController.cs`

Use **Excluir**, não "Remover do projeto". Em projetos SDK-style os arquivos `.cs` da pasta são incluídos no build automaticamente — remover do projeto apenas some com eles do Solution Explorer, e eles continuam compilando.

Parte 2 — Camada 04-Domain

O coração do sistema. Aqui mora a regra de negócio pura, sem nenhuma dependência externa.

06 Entidade `Cliente`

Crie a pasta `Entities` no `04-Domain` e, dentro dela, a classe `Cliente.cs`:

`04-Domain\Entities\Cliente.cs`

```
using System.ComponentModel.DataAnnotations;

namespace NoPrumo.Domain.Entities
{
    public class Cliente
    {
        public int Id { get; private set; }

        [Required(ErrorMessage = "O nome é obrigatório")]
        [StringLength(100, MinimumLength = 3)]
        public string Nome { get; private set; } = string.Empty;

        [Range(0, int.MaxValue, ErrorMessage = "A quantidade de obras não pode ser negativa")]
        public int ObrasSimultaneas { get; private set; }

        [Range(0, double.MaxValue, ErrorMessage = "O orçamento não pode ser negativo")]
        public decimal ValorOrçamento { get; private set; }

        // Construtor vazio e privado: só o EF Core usa, ao materializar do banco.
        private Cliente() { }

        public Cliente(string nome, int obrasSimultaneas, decimal valorOrçamento)
        {
            Nome = nome;
            ObrasSimultaneas = obrasSimultaneas;
            ValorOrçamento = valorOrçamento;
        }

        public void Atualizar(string nome, int obrasSimultaneas, decimal valorOrçamento)
        {
            Nome = nome;
            ObrasSimultaneas = obrasSimultaneas;
            ValorOrçamento = valorOrçamento;
        }
    }
}
```

POR QUE `PRIVATE SET`

Com `private set`, ninguém de fora consegue escrever `cliente.ValorOrçamento = -50`. As únicas portas de entrada são o construtor e o método `Atualizar()` — e é exatamente aí que a validação de negócio vai morar quando o projeto crescer. A entidade protege o próprio estado.

O construtor privado vazio existe só para o EF Core: ele precisa criar o objeto antes de preencher os campos, e consegue fazer isso por reflexão mesmo com os `set` privados.

07 Interface `IClienteRepository`

Crie a pasta `Interfaces` no `04-Domain` e a interface:

`04-Domain\Interfaces\IClienteRepository.cs`

```
using NoPrumo.Domain.Entities;

namespace NoPrumo.Domain.Interfaces
{
    public interface IClienteRepository
    {
        Task<IEnumerable<Cliente>> GetAllAsync();
        Task<Cliente?> GetByIdAsync(int id);
        Task AddAsync(Cliente cliente);
        void Update(Cliente cliente);
        void Delete(Cliente cliente);
        Task SaveChangesAsync();
    }
}
```

POR QUE A INTERFACE FICA NO DOMAIN

O Domain *declara o que precisa* — "me dê um jeito de salvar cliente" — sem saber se por trás existe MySQL, SQL Server ou um arquivo texto. Quem implementa é o `03-Infrastructure`, no passo 13. É isso que permite ao Domain não referenciar ninguém.

Duas decisões de assinatura

`Update` e `Delete` são `void`, não `Task`. No EF Core, esses métodos só marcam o estado da entidade no `ChangeTracker` — operação em memória, sem I/O. O banco é tocado uma única vez, no `SaveChangesAsync()`.

`GetAllAsync` devolve `IEnumerable`, não `List` nem `IQueryable`. `List` exporia `Add / Remove`, sugerindo que dá para alterar a coleção e salvar algo — e não dá. `IQueryable` seria pior: deixaria a consulta aberta, e a camada Application poderia pendurar `.Where()` que viraria SQL, ou seja, o Application montando query de banco. `IEnumerable` sinaliza dados já materializados na memória.

Parte 3 — Camada 02-Application

A camada que orquestra: recebe DTOs, conversa com o repositório, devolve DTOs.

08 DTOs de entrada e saída

Crie a pasta `DTOs` no `02-Application` com dois arquivos:

`02-Application\DTOs\ClienteInputDto.cs`

```
namespace NoPrumo.Application.DTOs
{
    public record ClienteInputDto(
        string Nome,
        int ObrasSimultaneas,
        decimal ValorOrcamento);
}
```

`02-Application\DTOs\ClienteOutputDto.cs`

```
namespace NoPrumo.Application.DTOs
{
    public record ClienteOutputDto(
        int Id,
        string Nome,
        int ObrasSimultaneas,
        decimal ValorOrcamento);
}
```

POR QUE NÃO USAR A ENTIDADE DIRETO NO CONTROLLER

Na entrada: se o endpoint recebesse `Cliente`, alguém poderia enviar `{"id": 999}` num POST e tentar escolher o próprio Id. O `Input` não tem `Id` — logo é impossível.

Na saída: no dia em que a entidade ganhar campos internos (custo real, margem, observações da equipe), eles não vazam para a resposta HTTP sem você decidir. O `Output` é um contrato separado que você controla.

Sobre `record`

Um `record` é uma classe — mesmo tipo de referência, mesma coisa em memória. O que muda é que o compilador escreve para você o construtor, as propriedades `init`, o `Equals`, o `GetHashCode`, o `ToString` e o suporte a `with`.

A diferença que mais aparece na prática é a **igualdade por valor**:

```
var a = new ClienteOutputDto(1, "Construtora X", 2, 50000);
var b = new ClienteOutputDto(1, "Construtora X", 2, 50000);

a == b // record: true | class: false
```

Com `class` , `==` compara endereço de memória: dois objetos distintos, ainda que idênticos, dão `false` . Com `record` , compara conteúdo.

Daí a regra: **DTO combina com `record` , entidade combina com `class`** . O DTO é *um valor* — dois clientes com os mesmos dados são o mesmo dado. Já `Cliente` é *uma identidade* — dois clientes de mesmo nome ainda são duas linhas diferentes no banco.

09 Interface `IClienteService`

02-Application\Interfaces\IClienteService.cs

```
using NoPrumo.Application.DTOS;

namespace NoPrumo.Application.Interfaces
{
    public interface IClienteService
    {
        Task<IEnumerable<ClienteOutputDto>> GetAllAsync();
        Task<ClienteOutputDto> GetByIdAsync(int id);
        Task<ClienteOutputDto> CreateAsync(ClienteInputDto dto);
        Task UpdateAsync(int id, ClienteInputDto dto);
        Task DeleteAsync(int id);
    }
}
```

Compare as duas interfaces já escritas:

Interface	Fala em	Mora em
<code>IClienteRepository</code>	<code>Cliente</code> (entidade)	04-Domain
<code>IClienteService</code>	<code>ClienteInputDto</code> / <code>ClienteOutputDto</code>	02-Application

O Service é a fronteira onde entidade vira DTO e vice-versa. Quem está de fora — o Controller — nunca enxerga a entidade `Cliente` .

POR QUE `CREATEASYNC` RECEBE INPUT E DEVOLVE OUTPUT

Entra `Input` porque o chamador manda apenas nome, obras e orçamento — quem decide o `Id` é o banco. Volta `Output` porque, depois do `SaveChangesAsync()` , o EF preenche o `Id` , e esse `Id` é justamente a informação nova que o chamador não tinha. Sem ele, ninguém saberia fazer o `GET /api/clientes/7` depois.

Já `UpdateAsync` retorna `Task` puro: no PUT o `Id` já é conhecido, não há informação nova a devolver. Por isso ele vira `204 No Content` no controller.

10 Implementação `ClienteService`

02-Application\Services\ClienteService.cs

```

using NoPrumo.Application.DTOS;
using NoPrumo.Application.Interfaces;
using NoPrumo.Domain.Entities;
using NoPrumo.Domain.Interfaces;

namespace NoPrumo.Application.Services
{
    public class ClienteService : IClienteService
    {
        private readonly IClienteRepository _repository;

        public ClienteService(IClienteRepository repository)
        {
            _repository = repository;
        }

        public async Task<IEnumerable<ClienteOutputDto>> GetAllAsync()
        {
            var clientes = await _repository.GetAllAsync();
            return clientes.Select(ToOutput);
        }

        public async Task<ClienteOutputDto> GetByIdAsync(int id)
        {
            var cliente = await _repository.GetByIdAsync(id);
            if (cliente == null)
                throw new KeyNotFoundException($"Cliente {id} não encontrado.");

            return ToOutput(cliente);
        }

        public async Task<ClienteOutputDto> CreateAsync(ClienteInputDto dto)
        {
            var cliente = new Cliente(dto.Nome, dto.ObrasSimultaneas, dto.ValorOrcamento);

            await _repository.AddAsync(cliente);
            await _repository.SaveChangesAsync();

            return ToOutput(cliente);
        }

        public async Task UpdateAsync(int id, ClienteInputDto dto)
        {
            var cliente = await _repository.GetByIdAsync(id);
            if (cliente == null)
                throw new KeyNotFoundException($"Cliente {id} não encontrado.");

            cliente.Atualizar(dto.Nome, dto.ObrasSimultaneas, dto.ValorOrcamento);

            _repository.Update(cliente);
            await _repository.SaveChangesAsync();
        }

        public async Task DeleteAsync(int id)
        {
            var cliente = await _repository.GetByIdAsync(id);
            if (cliente == null)
                throw new KeyNotFoundException($"Cliente {id} não encontrado.");
        }
    }
}

```

```

        _repository.Delete(cliente);
        await _repository.SaveChangesAsync();
    }

    // Tradutor: entidade -> DTO de saída. Um lugar só, usado por todos os métodos.
    private static ClienteOutputDto ToOutput(Cliente cliente) =>
        new(cliente.Id, cliente.Nome, cliente.ObrasSimultaneas, cliente.ValorOrcamento);
    }
}

```

Três pontos que valem atenção:

- **Depende de `IClienteRepository`, não de `ClienteRepository`**. O Service nunca importa nada do Infrastructure — inclusive porque o `02-Application` sequer referencia aquele projeto. Quem escolhe a implementação é o `Program.cs`.
- **throw em vez de retornar null**. O Service não sabe o que é HTTP, então não pode devolver "404". Ele lança `KeyNotFoundException` e o Controller traduz em `NotFound()`. Cada camada fala a própria língua.
- **CreateAsync converte depois do save**. Antes do `SaveChangesAsync()`, `cliente.Id` ainda vale `0`. É o EF que preenche o Id ao gravar.

Parte 4 — Camada 03-Infrastructure

O único lugar do projeto que sabe que Entity Framework existe.

11 Pacotes NuGet do EF Core

Botão direito no `03-Infrastructure` → **Gerenciar Pacotes NuGet** → aba **Procurar**:

Pacote	Versão
Pomelo.EntityFrameworkCore.MySql	8.0.3
Microsoft.EntityFrameworkCore.Tools	8.0.30

Cuidado com a versão. O NuGet sugere a mais recente (9.x ou 10.x) por padrão. O projeto é `net8.0`, então abra o dropdown **Versão** e escolha uma `8.0.x` — caso contrário o pacote exige um runtime maior que o do projeto e a compilação quebra. Qualquer patch `8.0.x` serve; mantenha o `Tools` em versão igual ou maior que a do EF Core que o Pomelo puxar.

Pomelo é o *provider* de MySQL: traduz o LINQ em SQL do MySQL. Ele já traz o

`Microsoft.EntityFrameworkCore` como dependência, então não é preciso instalar o EF Core separadamente.

Tools habilita os comandos `Add-Migration` e `Update-Database` no console — é ferramenta de desenvolvimento, não vai para o servidor.

12 AppDbContext

`03-Infrastructure\Data\AppDbContext.cs`

```
using Microsoft.EntityFrameworkCore;
using NoPrumo.Domain.Entities;

namespace NoPrumo.Infrastructure.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Cliente> Clientes { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Cliente>()
                .Property(cliente => cliente.ValorOrcamento)
                .HasColumnType("decimal(18,2)");
        }
    }
}
```

- `DbSet<Cliente> Clientes` declara que a entidade vira tabela. O nome da propriedade é o nome da tabela no banco.

- **DbContextOptions no construtor** — o contexto não escolhe a string de conexão; ela é injetada. Quem monta isso é o `Program.cs`. Assim o mesmo contexto serve para o MySQL real e para um banco em memória nos testes.
- **decimal(18,2)** — sem isso o EF não sabe quantas casas decimais reservar para `ValorOrcamento`, emite um aviso e arredonda de um jeito que você não escolheu. São 18 dígitos no total, 2 depois da vírgula.

13 ClienteRepository

03-Infrastructure\Repositories\ClienteRepository.cs

```
using Microsoft.EntityFrameworkCore;
using NoPrumo.Domain.Entities;
using NoPrumo.Domain.Interfaces;
using NoPrumo.Infrastructure.Data;

namespace NoPrumo.Infrastructure.Repositories
{
    public class ClienteRepository : IClienteRepository
    {
        private readonly AppDbContext _context;

        public ClienteRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Cliente>> GetAllAsync()
        {
            return await _context.Clientes.ToListAsync();
        }

        public async Task<Cliente?> GetByIdAsync(int id)
        {
            return await _context.Clientes.FindAsync(id);
        }

        public async Task AddAsync(Cliente cliente)
        {
            await _context.Clientes.AddAsync(cliente);
        }

        public void Update(Cliente cliente)
        {
            _context.Clientes.Update(cliente);
        }

        public void Delete(Cliente cliente)
        {
            _context.Clientes.Remove(cliente);
        }

        public async Task SaveChangesAsync()
        {
            await _context.SaveChangesAsync();
        }
    }
}
```

```
}  
}
```

ToListAsync() é o que materializa a consulta. Até essa linha existe apenas um **IQueryable** — uma query montada, não executada. O **ToListAsync()** dispara o **SELECT** de verdade e devolve objetos na memória, que é o que o **IEnumerable** do contrato prometia.

FindAsync difere de **FirstOrDefaultAsync** : ele primeiro verifica se a entidade já está no **ChangeTracker**, carregada nesta mesma requisição, e só vai ao banco se não estiver. Para busca por chave primária é a opção mais barata.

Parte 5 — Camada 01-Presentation

14 Pacote EFCore.Design

Botão direito no 01-Presentation → Gerenciar Pacotes NuGet:

Pacote	Versão
Microsoft.EntityFrameworkCore.Design	8.0.30

POR QUE ESTE PACOTE VAI NO PRESENTATION

Para gerar a migration, as ferramentas do EF precisam *rodar sua aplicação* e descobrir como o `AppDbContext` é construído — qual string de conexão, qual provider. Isso está no `Program.cs`, que fica aqui. O EF chama esse projeto de "projeto de inicialização".

Daí a divisão: `Tools` no 03-Infrastructure (onde vivem as migrations), `Design` no 01-Presentation (de onde vem a configuração).

15 String de conexão

01-Presentation\appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "server=127.0.0.1;port=3306;user=root;password=root;database=NoPrumoDB"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Ajuste `user` e `password` para os do seu MySQL. O banco `NoPrumoDB` ainda não existe — será criado pelo `Update-Database`, no passo 19. O nome aqui é a declaração de intenção.

Senha de banco em `appsettings.json` resolve para estudo, mas em projeto real vai para *User Secrets* ou variável de ambiente — esse arquivo acaba indo para o Git.

16 Program.cs

01-Presentation\Program.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.OpenApi.Models;
using NoPrumo.Application.Interfaces;
using NoPrumo.Application.Services;
```

```
using NoPrumo.Domain.Interfaces;
using NoPrumo.Infrastructure.Data;
using NoPrumo.Infrastructure.Repositories;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

// --- Banco de dados (EF Core + MySQL) ---
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(
        connectionString,
        ServerVersion.AutoDetect(connectionString)));

// --- Injeção de dependência ---
builder.Services.AddScoped<IClienteRepository, ClienteRepository>();
builder.Services.AddScoped<IClienteService, ClienteService>();

// --- Swagger ---
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "NoPrumo API",
        Version = "v1",
        Description = "Gestão de clientes e obras para pequenas empresas de construção."
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("/swagger/v1/swagger.json", "NoPrumo API v1");
    });
}

app.UseHttpsRedirection();

app.MapControllers();

app.Run();
```

O CORAÇÃO DA CLEAN ARCHITECTURE

```
builder.Services.AddScoped<IClienteRepository, ClienteRepository>();
builder.Services.AddScoped<IClienteService, ClienteService>();
```

Leia como: "*quando alguém pedir um `IClienteRepository`, entregue um `ClienteRepository`*". Este é o único ponto do sistema que amarra interface a implementação concreta — por isso o `ClienteService` pôde depender só da interface. Trocar MySQL por SQL Server um dia significa mudar essa linha, e nada mais.

AddScoped significa uma instância por requisição HTTP. Isso importa porque o `AppDbContext` também é scoped: dentro de um mesmo POST, o repositório e o contexto são os mesmos objetos, então o `AddAsync` e o `SaveChangesAsync` conversam com o mesmo `ChangeTracker`. Com `AddTransient`, cada chamada criaria um contexto novo e o `SaveChanges` não veria o que foi adicionado.

O `app.UseAuthorization()` que vinha no template foi removido: sem autenticação configurada ele não faz nada. Ele volta quando o projeto ganhar JWT.

17 ClientesController

01-Presentation\Controllers\ClientesController.cs

```
using Microsoft.AspNetCore.Mvc;
using NoPrumo.Application.DTOS;
using NoPrumo.Application.Interfaces;

namespace NoPrumo.Controllers
{
    [ApiController]
    [Route("api/clientes")]
    public class ClientesController : ControllerBase
    {
        private readonly IClienteService _service;

        public ClientesController(IClienteService service)
        {
            _service = service;
        }

        [HttpGet]
        [ProducesResponseType(typeof(IEnumerable<ClienteOutputDto>), StatusCodes.Status200OK)]
        public async Task<ActionResult<IEnumerable<ClienteOutputDto>>> GetTodos()
        {
            var clientes = await _service.GetAllAsync();
            return Ok(clientes);
        }

        [HttpGet("{id}")]
        [ProducesResponseType(typeof(ClienteOutputDto), StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status404NotFound)]
        public async Task<ActionResult<ClienteOutputDto>> GetPorId(int id)
        {
            try
            {

```

```

        var cliente = await _service.GetByIdAsync(id);
        return Ok(cliente);
    }
    catch (KeyNotFoundException)
    {
        return NotFound();
    }
}

[HttpPost]
[ProducesResponseType(typeof(ClienteOutputDto), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<ActionResult<ClienteOutputDto>> Criar(ClienteInputDto dto)
{
    var cliente = await _service.CreateAsync(dto);
    return CreatedAtAction(nameof(GetPorId), new { id = cliente.Id }, cliente);
}

[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> Atualizar(int id, ClienteInputDto dto)
{
    try
    {
        await _service.UpdateAsync(id, dto);
        return NoContent();
    }
    catch (KeyNotFoundException)
    {
        return NotFound();
    }
}

[HttpDelete("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> Remover(int id)
{
    try
    {
        await _service.DeleteAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException)
    {
        return NotFound();
    }
}
}

```

O contrato REST resultante

Verbo	Rota	Método	Resposta
GET	/api/clientes	GetTodos	200 + lista
GET	/api/clientes/5	GetPorId	200 ou 404
POST	/api/clientes	Criar	201 + header Location
PUT	/api/clientes/5	Atualizar	204 ou 404
DELETE	/api/clientes/5	Remover	204 ou 404

O controller faz apenas três coisas: recebe o DTO, chama o service, traduz o resultado em código HTTP. Zero regra de negócio, zero EF Core. É aqui que a `KeyNotFoundException` do Application vira `404` — a exceção era a língua de uma camada, o status é a língua da outra.

`[ApiController]` entrega de graça a validação das DataAnnotations que você pôs na entidade: se o JSON vier com nome vazio, o ASP.NET devolve `400` sozinho, sem passar pelo seu código.

`CreatedAtAction(nameof(GetPorId), ...)` é o que monta o header `Location: /api/clientes/7` na resposta do POST. Foi por isso que o `Criar` precisava do Id de volta.

Parte 6 — Banco de dados e validação

18 Gerar a migration

Confirme que o serviço do MySQL está rodando. Depois abra **Ferramentas → Gerenciador de Pacotes NuGet → Console do Gerenciador de Pacotes** e execute:

```
Add-Migration InitialCreate -Project 03-Infrastructure -StartupProject 01-Presentation
```

Parâmetro	Projeto	Papel
-Project	03-Infrastructure	onde está o DbContext e onde a pasta Migrations nasce
-StartupProject	01-Presentation	de onde vêm o Program.cs e a string de conexão

O EF carrega o Program.cs, descobre o ApplicationDbContext, compara o modelo atual com o que existe hoje (nada) e gera a pasta Migrations no 03-Infrastructure. **Ainda não toca no banco** — a migration é só o roteiro escrito.

O resultado do Up() deve descrever esta tabela:

```
Tabela Clientes
├─ Id                int, PK, auto_increment
├─ Nome              varchar(100), NOT NULL    ← veio do [StringLength] + [Required]
├─ ObrasSimultaneas int, NOT NULL
└─ ValorOrcamento  decimal(18,2), NOT NULL    ← veio do OnModelCreating
```

As DataAnnotations viraram tipos de coluna sozinhas. E o Down() sabe desfazer tudo — é o que permite voltar atrás com Remove-Migration.

19 Criar o banco

```
Update-Database -Project 03-Infrastructure -StartupProject 01-Presentation
```

Agora sim o EF conecta no MySQL. Como o NoPrumoDB não existe, ele cria o banco, cria a tabela Clientes e mais uma tabela chamada __EFMigrationsHistory — onde anota quais migrations já rodaram, para não aplicar duas vezes.

20 Testar o CRUD no Swagger

Aperte F5. O Swagger abre mostrando os cinco endpoints de /api/clientes. Teste nesta ordem:

1. POST /api/clientes → Try it out:

```
{
  "nome": "Construtora Silva",
  "obrasSimultaneas": 2,
```

```
"valorOrcamento": 450000.00  
}
```

Espera **201 Created**, com `id: 1` no corpo e o header `location: /api/clientes/1`.

2. **GET** `/api/clientes` → a lista deve trazer esse cliente.

3. **GET** `/api/clientes/1` → só ele. Depois teste `/api/clientes/99` → espera **404**.

4. **PUT** `/api/clientes/1` → mude `obrasSimultaneas` para `3` → espera **204 No Content**. Confirme com um **GET**.

5. **DELETE** `/api/clientes/1` → **204**. **GET** de novo → lista vazia.

Passando os cinco, o caminho completo está fechado: **Controller** → **Service** → **Repository** → **EF Core** → **MySQL**, com cada camada no seu lugar.

Apêndice A — Erros comuns

"Your target project '01-Presentation' doesn't match your migrations assembly '03-Infrastructure'"

O dropdown **Projeto padrão** do console está apontando para o `01-Presentation`. O EF chama de *migrations assembly* o projeto que contém o `DbContext` e de *target project* o que está selecionado no dropdown — e os dois precisam ser o mesmo, porque a pasta `Migrations` nasce junto do `DbContext`.

Solução: mude o dropdown para `03-Infrastructure`, ou passe `-Project 03-Infrastructure` no comando.

"Unable to resolve service for type 'DbContextOptions<AppDbContext>'"

O oposto do anterior: o **projeto de inicialização** virou o `03-Infrastructure`. Como ele é uma biblioteca de classes, não tem `Program.cs` — logo o EF não encontra o `AddDbContext(...)` e tenta construir o `AppDbContext` sem as opções.

Solução: botão direito no `01-Presentation` → *Definir como Projeto de Inicialização*, e passe `-StartupProject 01-Presentation` no comando.

Namespace duplicado ao colar código

Colar dentro do `namespace` que o Visual Studio já gerou produz algo assim:

```
namespace NoPrumo.Application.DTOS
{
    namespace NoPrumo.Application.DTOS // <- aninhado por engano
    {
        public record ClienteInputDto(...);
    }
}
```

O namespace real vira `NoPrumo.Application.DTOS.NoPrumo.Application.DTOS` e nenhum `using` encontra a classe. **Solução:** sempre **Ctrl+A** antes de colar num arquivo recém-criado.

Renomear um projeto

Renomear um projeto pelo Solution Explorer altera o `.csproj` e o nome exibido, mas **não renomeia a pasta no disco**. O `.slnx` passa a registrar um caminho em que pasta e arquivo têm nomes diferentes.

Como renomear por completo: remova o projeto da solution, renomeie a pasta pelo Explorador de Arquivos, e use *Adicionar* → *Projeto Existente* apontando para o `.csproj`.

Aviso MSB3277 no 05-Tests

Conflito entre versões do `Microsoft.EntityFrameworkCore` (por exemplo `8.0.13` vindo do Pomelo e `8.0.30` vindo do Tools). É apenas aviso, não impede a compilação. Resolve-se alinhando as versões dos pacotes quando o projeto de testes entrar em uso.

Apêndice B — Mapa final do projeto

```
NoPrumo/
|
├─ NoPrumo.slnx
|
├─ 01-Presentation/                                → NoPrumo
|   ├── Controllers/
|   |   └─ ClientesController.cs
|   ├── Properties/launchSettings.json
|   ├── appsettings.json                          ← string de conexão
|   ├── Program.cs                                ← DI, EF Core, Swagger
|   └─ NoPrumo.http
|
├─ 02-Application/                                → NoPrumo.Application
|   ├── DTOs/
|   |   ├── ClienteInputDto.cs
|   |   └─ ClienteOutputDto.cs
|   ├── Interfaces/
|   |   └─ IClienteService.cs
|   └─ Services/
|       └─ ClienteService.cs
|
├─ 03-Infrastructure/                             → NoPrumo.Infrastructure
|   ├── Data/
|   |   └─ AppDbContext.cs
|   ├── Migrations/
|   |   ├── 2026..._InitialCreate.cs
|   |   ├── 2026..._InitialCreate.Designer.cs
|   |   └─ AppDbContextModelSnapshot.cs
|   └─ Repositories/
|       └─ ClienteRepository.cs
|
├─ 04-Domain/                                     → NoPrumo.Domain
|   ├── Entities/
|   |   └─ Cliente.cs
|   └─ Interfaces/
|       └─ IClienteRepository.cs
|
└─ 05-Tests/                                     → NoPrumo.Tests
    (vazio nesta etapa)
```

O caminho de uma requisição POST

```
POST /api/clientes { nome, obrasSimultaneas, valorOrcamento }
|
▼
ClientesController.Criar(ClienteInputDto) [01-Presentation]
| valida DataAnnotations via [ApiController]
▼
ClienteService.CreateAsync(dto) [02-Application]
| new Cliente(...) → entidade
▼
ClienteRepository.AddAsync(cliente) [03-Infrastructure]
| _context.Clientes.AddAsync(...)
▼
SaveChangesAsync() → INSERT INTO Clientes ... [MySQL]
| EF preenche cliente.Id
▼
ToOutput(cliente) → ClienteOutputDto
|
▼
201 Created + Location: /api/clientes/1
```

Próximos passos possíveis

- **Entidade `obra`**. Hoje `ObrasSimultaneas` é um número solto no cliente. O passo natural é criar `Obra` como entidade própria — com endereço, prazo e status — e transformar o campo numa contagem da relação.
- **Autenticação JWT**. Adicionar `TokenService` no Infrastructure, `AuthController` no Presentation, e proteger POST/PUT/DELETE com `[Authorize]`. Aí o `app.UseAuthentication()` e o `app.UseAuthorization()` voltam ao `Program.cs`.
- **Testes no 05-Tests**. Com `Microsoft.EntityFrameworkCore.InMemory` dá para testar o `ClienteService` sem tocar no MySQL.
- **Middleware de exceção**. Hoje cada action tem seu `try/catch`. Um *exception handler* global centralizaria a tradução de `KeyNotFoundException` em `404`.